# System Integration and Deployment — Exam Theory Notes

**M.Sc. IT, Semester I | Based on the supplied 2026–27 syllabus**
Subject code: **MSCS33101** (SEC: System Integration and Deployment)

These notes cover all four syllabus units: Introduction to Version Control; Git Fundamentals, Branching and Merging; GitHub Basics; and System Integration & Deployment.

---

## Unit 1 — Introduction to Version Control

### 1.1 What is a Version Control System?

A **Version Control System (VCS)** is software that records and manages changes to files over time. It enables developers to review earlier versions, identify who changed what and when, compare revisions, and collaborate without simply overwriting one another's work.

A saved revision is commonly called a **version**, **revision**, or **commit**, depending on the system. A VCS can track source code and other text-based project files; many systems can also store binary files, though comparing binary changes is often less informative.

### 1.2 Why version control is useful

- **Change history:** Maintains a record of project changes and their authors.
- **Recovery:** Allows restoration of earlier versions after mistakes or data loss.
- **Collaboration:** Lets multiple people work on a shared project in a controlled manner.
- **Branching:** Supports parallel work on features, experiments, or fixes.
- **Comparison and auditing:** Shows differences between versions and supports traceability.
- **Reduced risk:** Makes it safer to test changes without damaging the stable version.
- **Backup and sharing:** A remote repository can provide an additional copy and a collaboration point.

Version control does **not** replace good backups, code review, testing, or security controls. It records changes; it does not guarantee that those changes are correct or safe.

### 1.3 Centralized vs Distributed VCS

| Feature | Centralized VCS (CVCS) | Distributed VCS (DVCS) |
|---|---|---|

| | | |
|---|---|---|
| Main model | A central server stores the main history | Each clone normally contains the repository history locally |
| Examples | Subversion (SVN), CVS | Git, Mercurial |
| Working offline | Usually limited; server access is often needed for history-changing operations | Many operations, including commits and history inspection, work offline |
| Failure risk | Central server failure can block collaboration and may threaten the only full history copy | Multiple clones can contain repository history; a remote outage does not prevent all local work |
| Collaboration | Users interact with the central repository | Changes can be exchanged among remotes and repositories |
| Complexity | Often conceptually simpler | More flexible, but introduces concepts such as local branches and distributed histories |

**Key distinction:** "Centralized" or "distributed" describes where repository history and collaboration operations are organized—not whether a project has a website or server. GitHub is a hosting and collaboration service; Git itself is a distributed VCS.

### Likely exam questions — Unit 1

1. Define a Version Control System and explain its major benefits.
2. Compare centralized and distributed version control systems.
3. Explain how version control supports teamwork and recovery.
4. Is GitHub itself a version control system? Explain the difference between Git and GitHub.

---

# Unit 2 — Git Fundamentals, Branching and Merging

## 2.1 Introduction to Git

**Git** is a free, open-source, distributed version control system. It records project history as a sequence of commits. Each commit identifies a snapshot of tracked project content together with metadata such as its author, time, message, and relationship to earlier commits.

Git is distributed because a clone normally contains the project files and repository history. Developers can create commits and branches locally, then share them with another repository (often a hosted remote).

## 2.2 Installing and configuring Git

Git is installed using an operating-system package manager or an installer. After installation, a user can configure identity information used in commit metadata. Common examples are:

```Bash
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

`--global` applies the setting for the current user across repositories; without it, a setting can be applied only to the current repository. Configuration identifies the author in history; it is not, by itself, authentication to a remote service.

## 2.3 Git architecture and workflow

The basic Git workflow has three main areas:

1. **Working directory:** The checked-out project files where the user edits content.
2. **Staging area (index):** A preparation area that records which changes will be included in the next commit. It lets a user select changes, rather than committing every modification at once.
3. **Repository (local database):** Stores committed history and Git metadata.

A typical cycle is:

```Plain Text
Edit files → inspect changes → stage selected changes → commit → share with rem
```

The staging area is a key distinction from a simple “edit then save” workflow: it defines the proposed contents of the next commit. Git also records file states such as **untracked**, **modified**, **staged**, and **committed**.

## 2.4 Creating repositories

A **repository** (repo) is the project plus the metadata that tracks its history.

- `git init` creates a new local repository in the current directory.

- `git clone <url>` creates a local copy of an existing repository, normally including its history and a remote reference to the source.

Initialization does not automatically create a hosted repository. A local repository can later be connected to a remote repository.

## 2.5 Core commands

| Command | Purpose |
| --- | --- |
| `git init` | Initialize a new local repository |
| `git clone <url>` | Copy an existing repository to the local machine |
| `git status` | Show branch and working-tree/staging status |
| `git add <file>` | Stage a file’ s current changes for the next commit; `git add .` stages changes under the current location, subject to ignore rules |
| `git commit -m "message"` | Record staged changes as a commit |
| `git log` | Display commit history |

Example:

Bash

```
git init
git status
git add README.md
git commit -m "Add project overview"
git log
```

A good commit message is concise, describes the intent of the change, and is specific enough to make history understandable. Check `git status` before committing to confirm what is staged.

## 2.6 Branching and branch management

A **branch** is a movable reference to a line of development. Branches let developers work on independent features or fixes without immediately changing the main development line. In Git, creating and switching branches is generally inexpensive.

Typical branch workflow:

Bash

```
git branch                          # list local branches
git switch -c feature/login         # create and switch to a branch
git status
git add .
git commit -m "Implement login"
git switch main                     # switch back to main
git merge feature/login             # integrate feature branch
```

Older material may show `git checkout` for switching or creating branches. `git switch` is the newer, more focused command for switching branches; both concepts may appear in textbooks.

## 2.7 Merging and merge conflicts

**Merging** integrates changes from one branch into another. For example, merging a completed feature branch into `main` incorporates that work into the target branch. Git may perform a **fast-forward merge** when the target has not diverged, or create a **merge commit** when histories have diverged and the changes can be combined.

A **merge conflict** occurs when Git cannot automatically determine how to combine concurrent changes—for example, when two branches modify the same lines incompatibly, or one deletes a file that another modifies.

General conflict-resolution process:

1. Run the merge and identify files reported as conflicted.
2. Open each file and inspect conflict markers ( `<<<<<<<` , `=======` , `>>>>>>>` ).
3. Decide the correct combined content; do not blindly keep one side.
4. Remove the conflict markers and save the resolved file.
5. Stage the resolved file with `git add` .
6. Complete the merge with a commit if Git requests one.
7. Run relevant tests and inspect the resulting history/status.

A conflict is not necessarily an error in Git; it is a request for a person to make a decision Git cannot infer safely.

## 2.8 Branching strategy and best practices

A **branching strategy** defines how a team names, creates, reviews, and integrates branches. Common approaches include:

- **Feature branches:** A separate branch for each feature or task; merge after review and testing.
- **Short-lived branches:** Keep work branches small and integrate them frequently to reduce divergence and complex conflicts.
- **Release branches:** Stabilize a planned release separately when a team needs a dedicated release process.
- **Hotfix branches:** Isolate urgent production fixes for controlled validation and release.
- **Trunk-based development:** Integrate small changes into a shared mainline frequently, often using short-lived branches and automated tests.

Good practice includes using meaningful branch names, making small focused commits, pulling/reconciling changes before integration, reviewing changes, running tests, and avoiding long-lived divergence. A strategy should fit the team and release process; no single strategy is best for every project.

### Likely exam questions — Unit 2

1. Explain Git’ s architecture and the roles of the working directory, staging area, and repository.
2. Describe the basic Git workflow with suitable commands.
3. Differentiate `git init` and `git clone`.
4. What is a branch? Explain branch creation and merging.
5. Define a merge conflict and describe its resolution steps.
6. Explain any three branching strategies and state their benefits.
7. Why is the staging area useful?

---

## Unit 3 — GitHub Basics

### 3.1 What is GitHub?

**GitHub** is a web-based platform for hosting Git repositories and supporting collaborative software development. It builds on Git and provides remote repository hosting plus collaboration features such as pull requests, issues, reviews, project coordination, and automation integrations.

**Git vs GitHub:** Git is the version control software; GitHub is a platform that hosts Git repositories and adds collaboration and project features. Git can be used without GitHub, and GitHub repositories are managed using Git.

## 3.2 Creating an account

A GitHub account provides an identity for using the service. Users should protect accounts with strong, unique credentials and any available multi-factor authentication. Account access and permissions should be managed carefully, especially for organization repositories. Account creation details can change; the central exam concept is that a user identity enables authorized interaction with hosted repositories.

## 3.3 Creating repositories on GitHub

A GitHub repository can be created through the website. Common setup choices include repository name, description, visibility (public or private), and whether to initialize with files such as a README or `.gitignore`.

- **Public repository:** Generally visible to the public; visibility does not automatically grant everyone write permission.
- **Private repository:** Access is restricted to authorized users and collaborators.
- **README:** Describes the project and is often displayed on the repository's front page.
- **`.gitignore`:** Specifies files Git should not normally track, such as generated build outputs or local secrets. It does not remove a secret already committed to history.

## 3.4 Connecting local repositories to GitHub

A local repository can be connected to a hosted repository using a **remote**, which is a named reference to another repository. `origin` is the conventional default name for the primary remote, though the name is configurable.

Example for a repository that already exists locally:

Bash

```
git remote add origin <repository-url>
git branch -M main
git push -u origin main
```

The exact branch name and authentication setup depend on the repository and account configuration. If cloning from GitHub, Git typically configures the source remote automatically.

## 3.5 Pushing and pulling changes

- **Push:** Sends local commits to a remote repository, subject to access permissions and remote history rules. Example: `git push origin main`.

- **Fetch:** Downloads remote objects and references without necessarily integrating them into the current branch. Example: `git fetch origin`.
- **Pull:** Usually fetches remote changes and integrates them into the current branch, commonly by merge (or by rebase if configured). Example: `git pull origin main`.

A simple collaboration cycle is:

```
Plain Text
Clone → create branch → edit → stage and commit → push branch → request review/
```

Before pushing, the developer should check the current branch, review the changes, and ensure the remote and local histories can be reconciled. A push may be rejected if the remote has commits absent locally; the developer should first fetch and integrate the remote work safely.

### 3.6 Forking repositories

A **fork** is a server-side copy of a repository under another account or namespace. Forking is common in open-source collaboration: a contributor forks a project, clones the fork, makes changes, pushes to their fork, and proposes those changes to the original project—often using a **pull request**.

A **pull request (PR)** is a platform-based proposal to review and integrate changes from one branch or repository into another. It supports discussion, review, automated checks, and approval workflows. A PR is not the same as a Git merge command, though it commonly leads to a merge.

### Likely exam questions — Unit 3

1. Explain GitHub and distinguish it from Git.
2. Describe how a local repository is connected to a GitHub repository.
3. Differentiate push, fetch, and pull.
4. What is forking? Explain its role in open-source contribution.
5. Explain pull requests and their benefits.
6. What are repository visibility and `.gitignore`? Explain their purpose.

---

## Unit 4 — System Integration and Deployment

### 4.1 Integration

**System integration** is the process of combining software components, code contributions, services, or subsystems so they work together as a complete system. In software development, integration includes merging changes and verifying that components remain compatible.

## 4.1.1 Continuous Integration (CI)

**Continuous Integration** is a development practice in which developers integrate small changes into a shared codebase frequently. Each integration is checked by an automated process—commonly build steps and tests—so defects and incompatibilities are detected early.

Typical CI sequence:

```
Plain Text

Code change → push / pull request → checkout code → install dependencies → buil
```

Benefits include early defect detection, faster feedback, reduced integration risk, repeatable validation, and improved confidence in the shared codebase. CI works best when tests are maintained, changes are small, and failures are fixed promptly. CI does not mean “testing happens only at the end,” nor does it guarantee bug-free software.

## 4.1.2 GitHub Actions overview

**GitHub Actions** is GitHub’s automation and CI/CD service. Workflows are defined in YAML files—typically under `.github/workflows/`—and can run in response to repository events or on a schedule/manual trigger.

Core concepts:

- **Workflow:** The complete automated process described in a YAML file.
- **Event/trigger:** The occurrence that starts a workflow, such as a push or pull request.
- **Job:** A group of steps executed on a runner; jobs can have dependencies or run in parallel.
- **Runner:** The machine/environment that executes a job.
- **Step:** An individual command or reusable action within a job.
- **Action:** A reusable unit of automation, such as checking out code or setting up a runtime.

A simplified conceptual workflow:

```
YAML
```

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Install dependencies, build, and run tests here"
```

The example illustrates structure only; real workflows must use commands appropriate to the application and should follow secure action and permissions practices.

### 4.1.3 Creating CI workflows

A basic workflow design involves:

1. Decide which events should trigger validation (for example, pushes and pull requests).
2. Define the runner environment.
3. Check out the repository code.
4. Set up required language/runtime versions.
5. Install dependencies reproducibly.
6. Run formatting, static analysis, build, and automated tests as appropriate.
7. Report success or failure clearly; prevent unsafe integration when required checks fail.

Good workflows should be repeatable, reasonably fast, and secure. Pin or carefully manage dependencies and actions, avoid exposing credentials in logs, grant only necessary permissions, and use secrets for sensitive values.

### 4.1.4 Automated testing

**Automated testing** uses software to execute predefined checks and compare actual results with expected behavior. Common levels include:

- **Unit tests:** Check small individual functions or components.
- **Integration tests:** Check interactions between components or services.
- **System/end-to-end tests:** Check larger workflows through the integrated application.
- **Acceptance tests:** Evaluate whether specified user or business requirements are met.

Automated tests provide repeatable feedback and can run on each change. They complement—not eliminate—manual exploratory testing, code review, security review, and usability evaluation. A passing test suite only demonstrates that the covered checks passed under the tested conditions.

### 4.1.5 Build automation

**Build automation** uses tools and scripts to perform repeatable tasks that turn source code into a usable artifact or verify it. Tasks may include dependency installation, compilation, bundling, code generation, packaging, and test execution.

Benefits include consistency, fewer manual errors, faster feedback, reproducibility, and easier integration with CI/CD pipelines. Builds should use controlled dependencies and configuration; “it works on my machine” problems often arise from differences in tools, environment, or dependency versions.

## 4.2 Deployment

**Deployment** is the process of making a software version available in a target environment for use or evaluation. A deployment process may include packaging, configuration, releasing, and verification.

### 4.2.1 Continuous Delivery vs Continuous Deployment

- **Continuous Delivery:** Changes are built, tested, and kept ready for release; production release may require a human approval or explicit decision.
- **Continuous Deployment:** Every change that passes the required automated checks is automatically released to production, without a separate manual release approval.

Both practices automate the route from code change toward release. The defining distinction is whether production deployment is automatic after checks or held for an explicit release decision.

### 4.2.2 Deployment pipelines

A **deployment pipeline** is an automated sequence that moves a change through validation and release stages. A representative pipeline is:

```
Plain Text

Commit → build → tests → package/artifact → deploy to test/staging → verify/app
```

A pipeline can stop a change from advancing when a required check fails. Artifacts should be traceable to a source commit, and each stage should provide clear logs and outcomes. Promotion of the same verified artifact between environments helps reduce “built differently for production” risks.

### 4.2.3 Environment management

Environments provide separate contexts for development, validation, and live use:

- **Development:** Used for active coding and early checks; changes may be unstable.
- **Testing/QA:** Used for systematic verification against requirements and expected behavior.
- **Staging/pre-production:** A production-like environment used for final validation before release.
- **Production:** The live environment used by real users or operations.

Environment management includes configuration, access controls, secrets, data handling, dependencies, and deployment procedures. Environments should be sufficiently consistent to make test results meaningful, while sensitive production data and credentials must be protected. Keep environment-specific configuration separate from application code and never commit secrets to a repository.

### 4.2.4 GitHub Actions for deployment

GitHub Actions can automate deployment when specified events and checks occur. A deployment workflow may build and test an application, then deploy to a hosting platform or server using an approved action or command. Secure design should:

- Use repository or environment secrets for credentials and avoid printing them.
- Apply least privilege to workflow permissions.
- Restrict production deployment to approved branches/environments and, where needed, require approval.
- Separate CI validation from production release logic when appropriate.
- Keep logs, artifacts, and deployment outcomes traceable.

The precise deployment steps depend on the hosting platform and application. A workflow file coordinates the process; it does not itself provide a hosting service.

### 4.2.5 Monitoring and rollback strategies

**Monitoring** observes application health and behavior after deployment. It can include availability, error rates, response time, resource use, logs, traces, and business-level signals. Monitoring helps teams discover problems that automated pre-release tests did not reveal.

A **rollback** restores a previous known-good release or configuration when a deployment causes unacceptable problems. Effective rollback planning includes versioned artifacts, deployment history, tested recovery procedures, backups or data migration plans, and clear decision criteria.

Rollback is not always as simple as redeploying old code: database schema changes, data transformations, or external side effects may make reversal difficult. Prefer compatible, incremental changes and plan forward fixes or recovery when a direct rollback is unsafe.

Common deployment approaches:

- **Rolling deployment:** Replace instances gradually; reduces downtime but requires compatible versions during transition.
- **Blue-green deployment:** Maintain old and new environments, then switch traffic; supports rapid switch-back but may require duplicated resources.
- **Canary release:** Send a small portion of traffic to the new version, observe results, then expand or roll back; limits initial exposure but needs reliable monitoring and traffic control.

### Likely exam questions — Unit 4

1. Define CI and explain its workflow and benefits.
2. Explain GitHub Actions and its main components: workflow, event, job, runner, step, and action.
3. Describe how an automated CI workflow is created.
4. Explain automated testing and distinguish unit, integration, and end-to-end tests.
5. What is build automation? Explain its benefits.
6. Differentiate Continuous Delivery and Continuous Deployment.
7. Explain the stages in a deployment pipeline.
8. Describe development, testing, staging, and production environments.
9. Explain how GitHub Actions can support deployment securely.
10. What are monitoring and rollback? Explain rolling, blue-green, and canary deployment strategies.

---

## High-value comparisons for revision

| Comparison | Remember this distinction |
|---|---|
| Git vs GitHub | Git is the distributed version-control tool; GitHub hosts repositories and adds collaboration features. |

| | |
|---|---|
| CVCS vs DVCS | Central server-centered history vs complete local repository history in normal clones. |
| Working directory vs staging area vs repository | Edit files vs select next-commit content vs store committed history. |
| `git fetch` vs `git pull` | Fetch downloads remote updates; pull fetches and integrates them into the current branch. |
| Branch vs fork | Branch is a line of development within a repository; fork is a server-side copy under another namespace/account. |
| Merge vs pull request | Merge is the history integration operation; a pull request is a review/proposal workflow on a hosting platform. |
| CI vs CD | CI validates frequent integrations; CD automates readiness/release, with delivery allowing a release decision and deployment releasing automatically. |
| Testing vs monitoring | Testing checks behavior under planned conditions; monitoring observes the running system after deployment. |

# Exam answer-writing framework

For a 5–10 mark theory answer:

1. **Define** the key term in one or two direct sentences.
2. **Explain** its components or process in ordered points.
3. **Add a diagram or workflow** where useful (for example, Git’ s three areas or a CI/CD pipeline).
4. **State benefits and limitations** rather than listing benefits only.
5. **Give a short example** such as a feature branch or a push-triggered test workflow.
6. **Conclude** with the main purpose or distinction.

For comparison questions, define both concepts first, then compare them across consistent points, and finish with their relationship or appropriate use.

# One-minute final revision

- VCS records change history; centralized VCS relies on a central repository, while Git is distributed.
- Git’ s flow: working directory → staging area → local repository → remote (when sharing).
- `init` starts a repository; `clone` copies an existing one; `add` stages; `commit` records; `push` shares; `pull` fetches and integrates.
- Branches isolate work; merges integrate it; conflicts require deliberate human resolution.
- GitHub hosts Git repositories; a fork is a copy, and a pull request proposes reviewed integration.
- CI repeatedly builds/tests integrated changes. GitHub Actions executes event-triggered workflows made of jobs, runners, and steps.
- Deployment pipelines automate validation and delivery. Continuous Delivery can require release approval; Continuous Deployment automatically releases passing changes.
- Separate environments, protect secrets, monitor releases, and plan rollback before production deployment.